

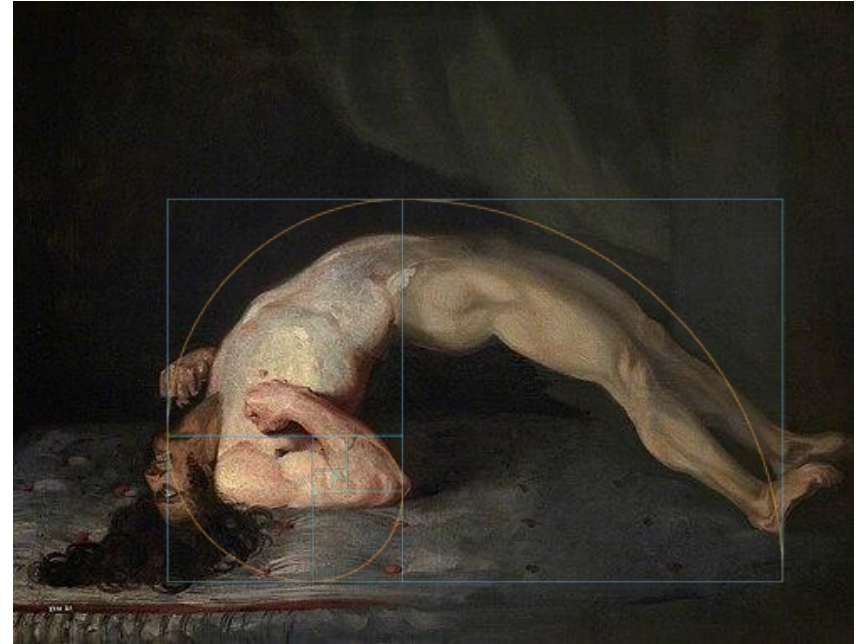
# Isometric Resistance as Control Signal in Hyperkinesia

Hackaques

Hannah Ryu, Edric Tsang, Eric Han and Derrick Tang  
Neurohackathon 2025 | May 18th 2025

# Movement disorders may involve involuntary contractions

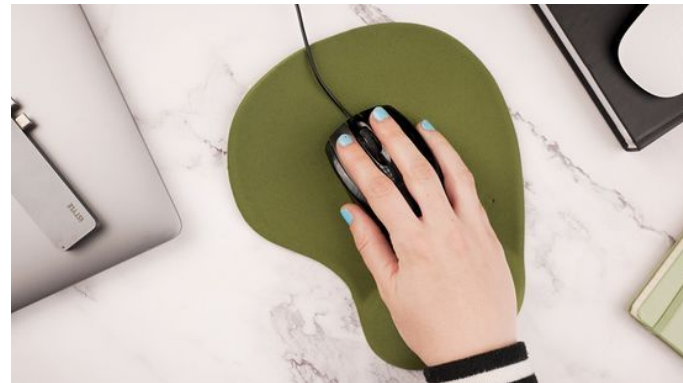
- Hyperkinesia: cervical dystonia, Tourette's syndrome, Huntington's disease, tetanus
- Vary in severity: frequency and magnitude of contraction
- Hyperkinetic disorders make motor tasks difficult



Patient suffering from tetanus - Charles Bell (1809)

# Imagine...

- Controlling a mouse with involuntary twitches
  - Cursor overshoots or undershoots
  - Smooth motions impossible
  - Unintentional clicking
  - Frustrating!
- Highly irregular frequency and magnitude of twitches is a design challenge
- Hyperkinesia can be partially treated with botox, compensated with software
- Our strategy: utilize subtle motor activation that persists during twitching



# Isometric contractions can counter involuntary contractions



Principal Skinner's isometric exercise - *The Simpsons*

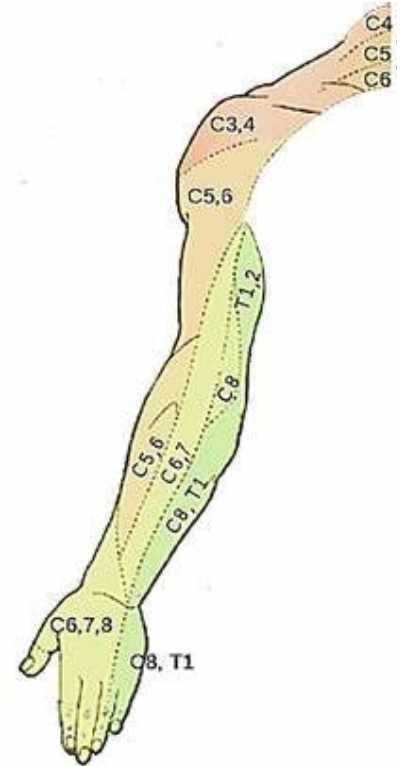
- Isometric contraction feels effortful but no change in muscle length
  - Planks, wall sits, hamstring stretch
- Subtle and not visibly observable
- Requires minimal control of skeletal muscles

# Motivation: non-invasive myoelectric control signal

- Test feasibility of isometric resistance (subtle and low fatigue) as a control signal in moderate to severe hyperkinesia during twitches
- Expand accessibility for hyperkinesia with very low motor control
- Invasive and non-invasive Brain-Computer Interface (BCI)'s and myoelectric interfaces have proven successful
  - Restoring motor function
  - Granting control of computers/robotic prosthetics
- Limited hardware availability: we grabbed last

# Simulation, recording, and data processing

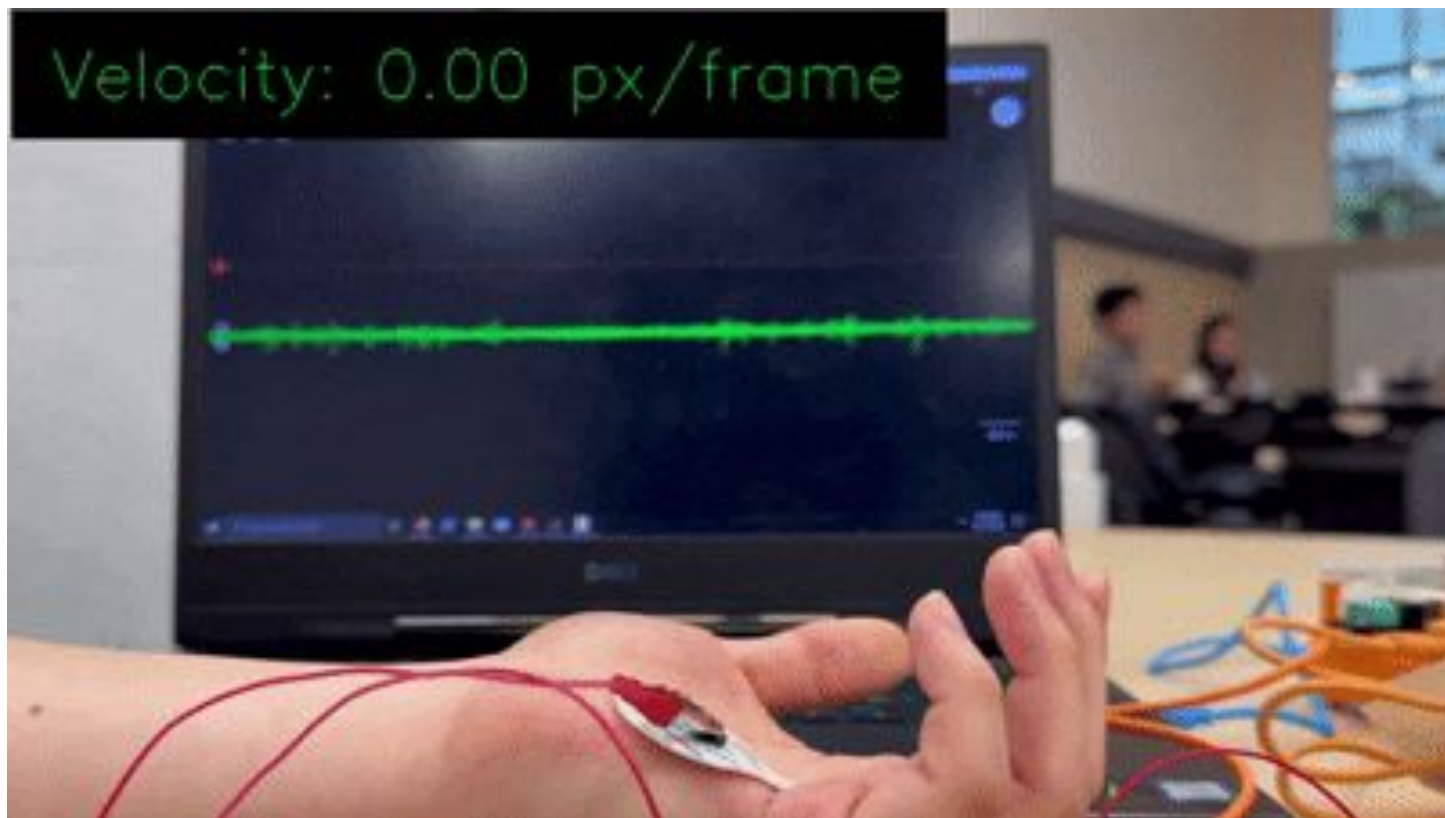
- TENS 3000 stimulator:
  - Simulate involuntary finger twitches in hyperkinesia
  - Send stimulation timing to Spikerbox
- Backyard Brains Muscle Spikerbox:
  - Record all muscle activity and inactivity
  - Butterworth filter 20Hz-450Hz
  - Align to TENS stimulation
- Phone camera
  - Record all hand movement and lack of movement
  - Use DeepLabCut to quantify movement



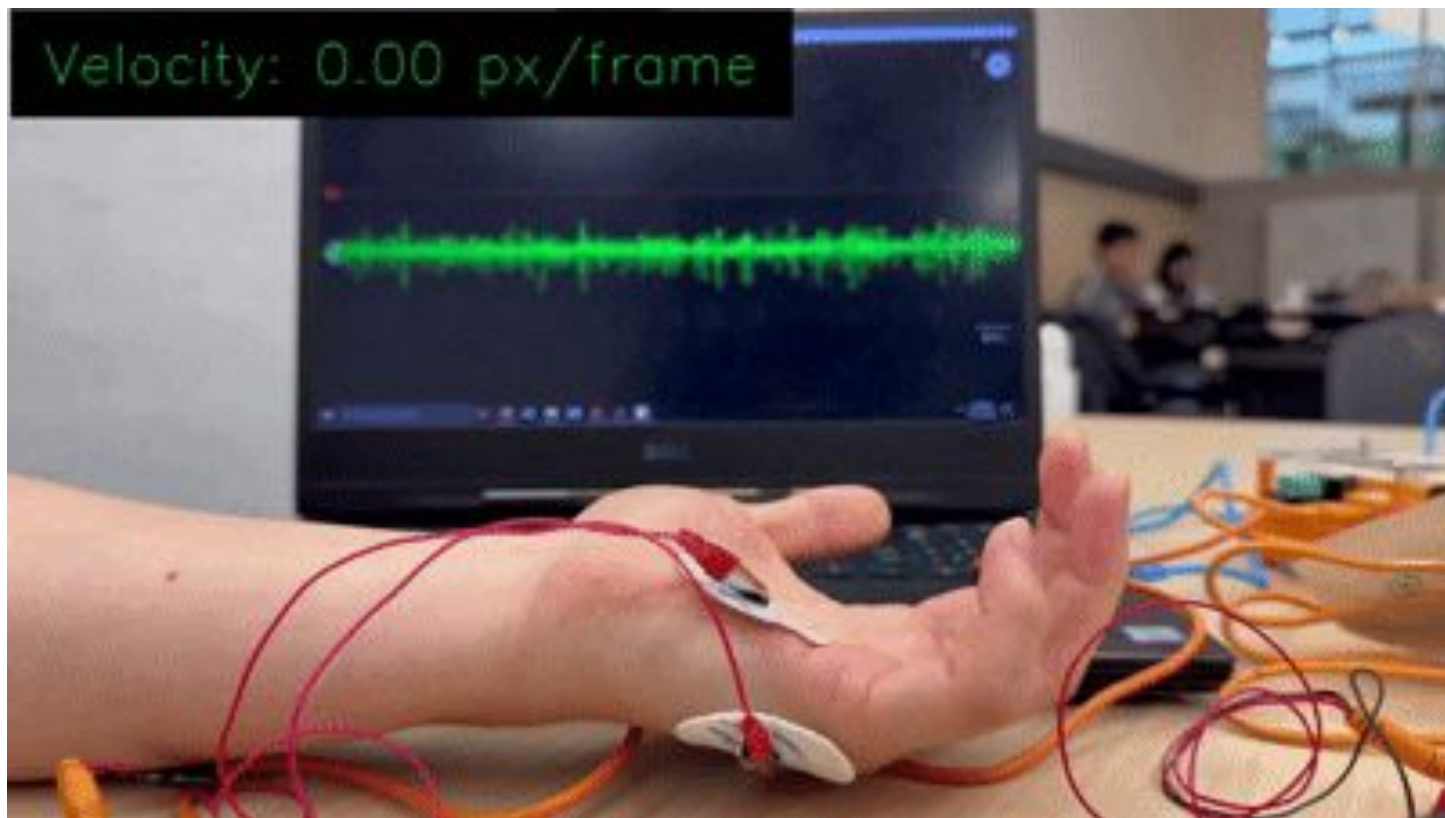
# Experimental Design

|                | No Isometric Resistance                                                                              | Isometric Resistance                                                                                                 |
|----------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| No Stimulation |  Rest / Baseline    |  Attempted Movement (No Stimulus) |
| Stimulation    |  Involuntary Twitch |  Intentional Suppression          |

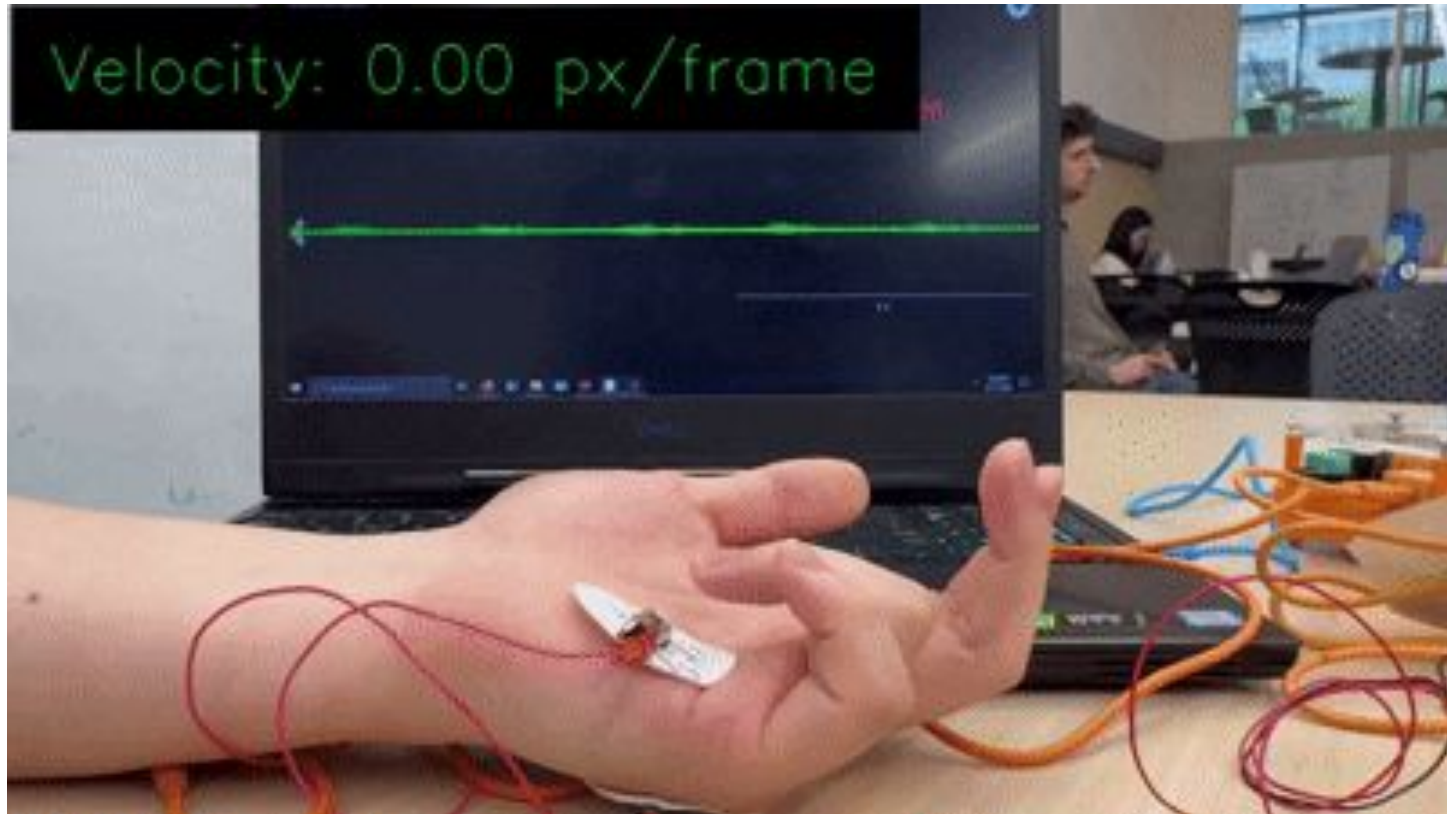
# No Stimulation | No Resistance



No Stimulation | Resistance (tensing up)

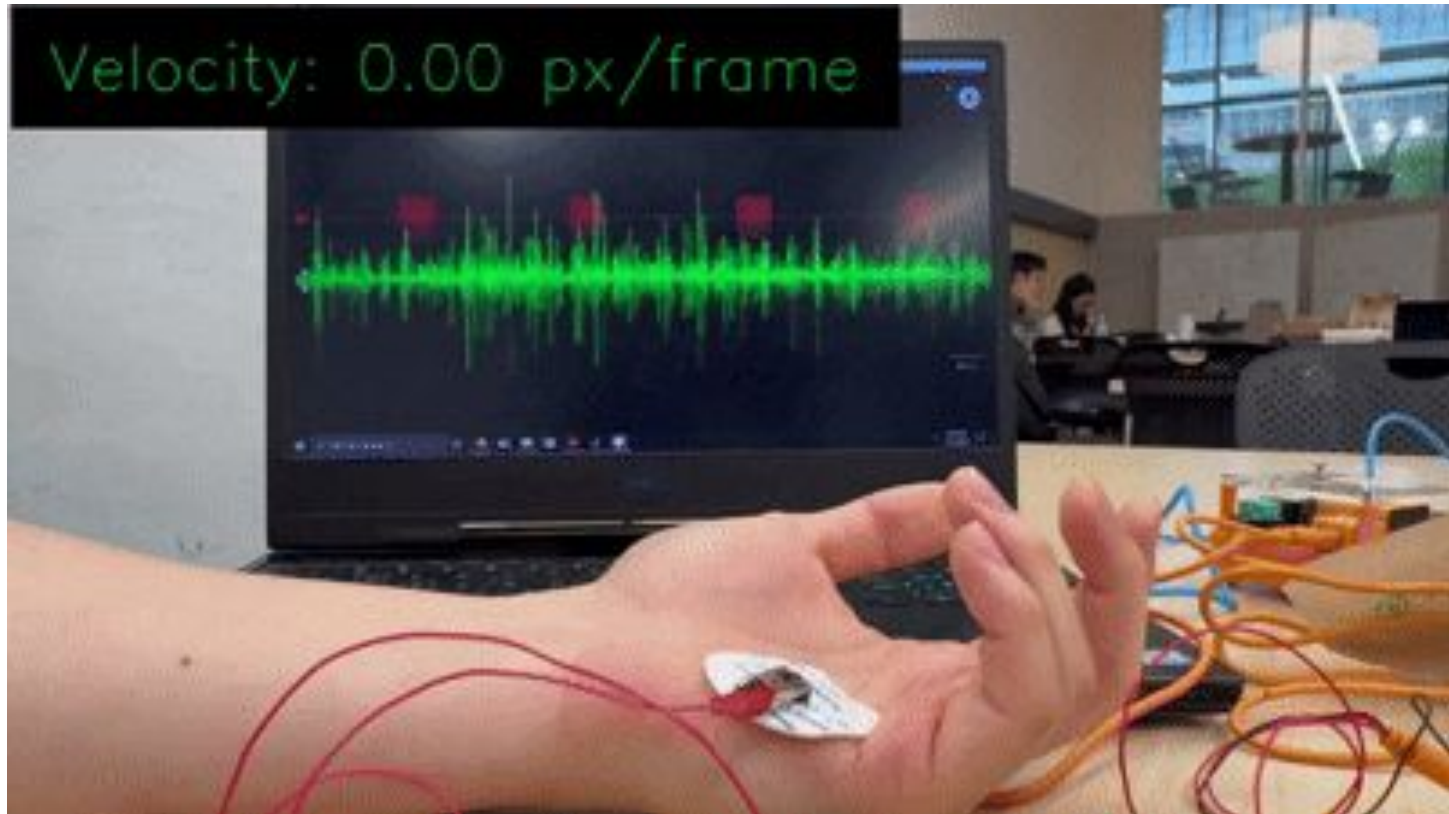


## Stimulation | No Resistance



Note: green channel not scaled correctly

# Stimulation | Resistance



# Experimental Design

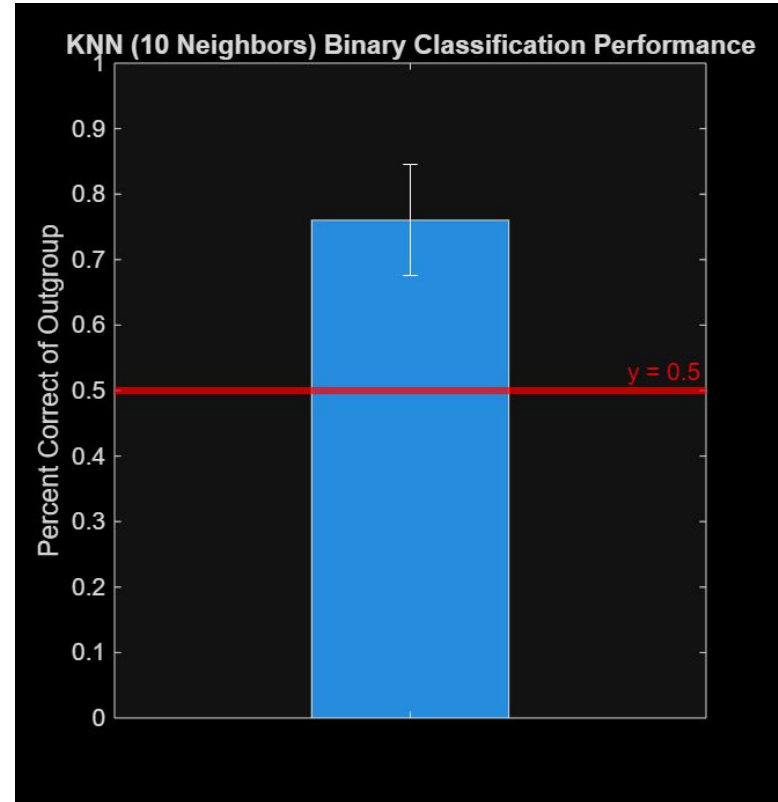
|                | No Isometric Resistance                                                                              | Isometric Resistance                                                                                                 |
|----------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| No Stimulation |  Rest / Baseline    |  Attempted Movement (No Stimulus) |
| Stimulation    |  Involuntary Twitch |  Intentional Suppression          |

# Resistance during stimulation is perceptible



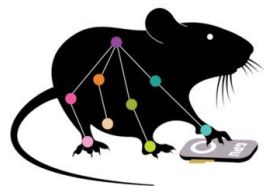
# Binary classification using KNN above chance levels

- 80-20 train test split
- 20 randomized repetitions
- +/-1 SD shown
- Null rejected in Z-test ( $p < 0.001$ )



# Conclusion and Application (Future Direction)

- We started with a hypothesis about resisting involuntary movements
- Isometric resistance to hyperkinetic twitches can be used as a control signal via surface EMG
- More exploratory analysis, better classification methods
- Application of the myoelectric control signal can be anything:
  - Highly scalable: EMG abundance
  - Brain stimulation switch
  - Computer cursor x-y movements (must add one EMG electrode set)
  - Prosthetic with finger actuation (must add many EMG electrode sets)
  - Privacy must be protected in connected devices (ethical consideration)



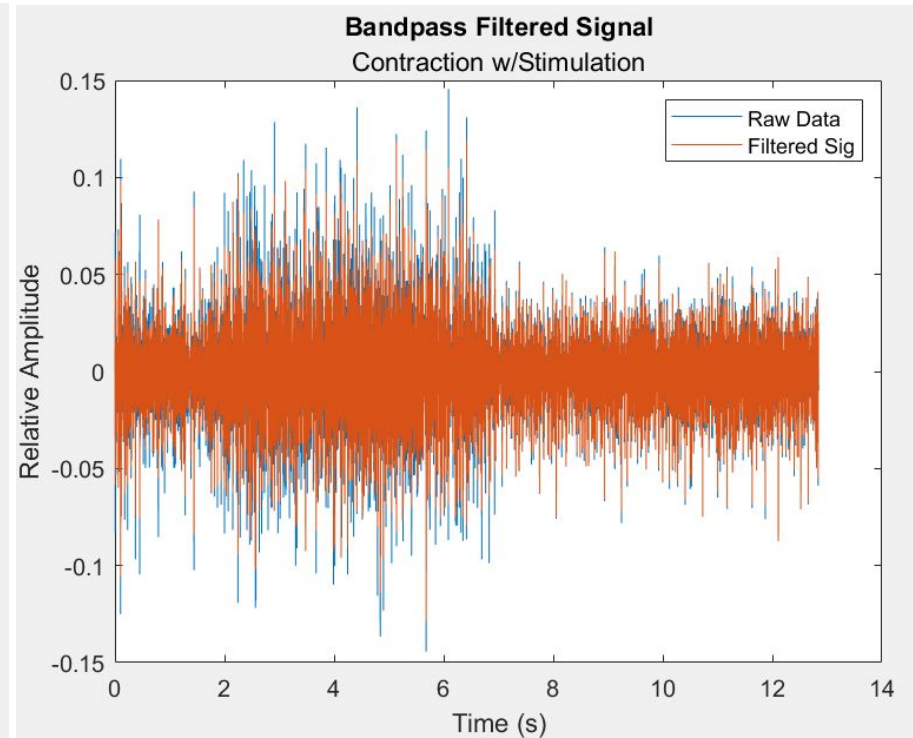
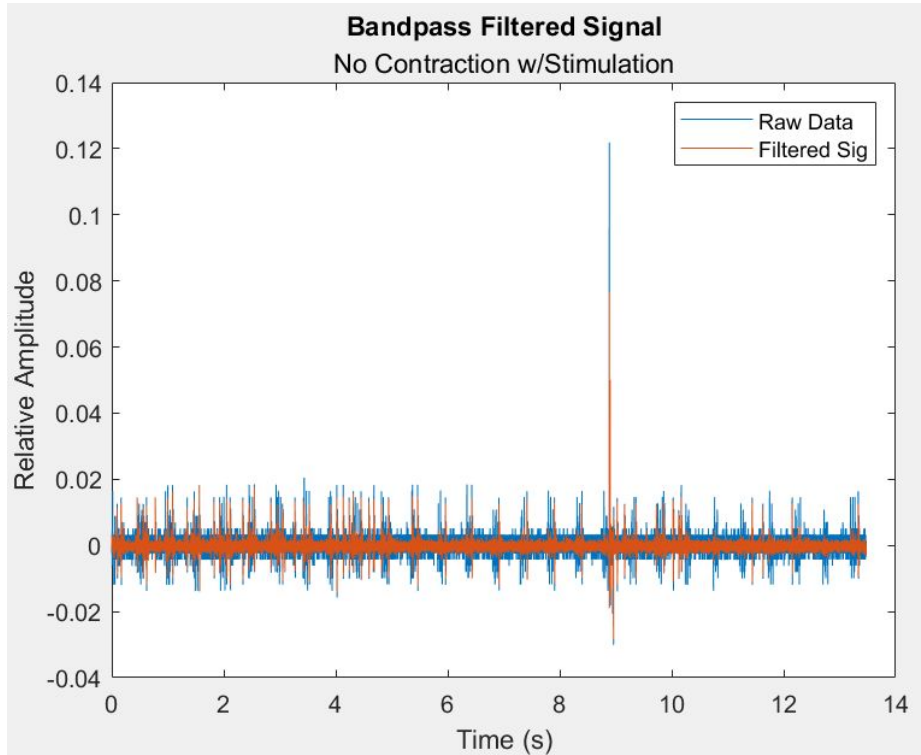
DeepLabCut<sup>TM</sup>:  
a software package for  
animal pose estimation

# Questions/Feedback?

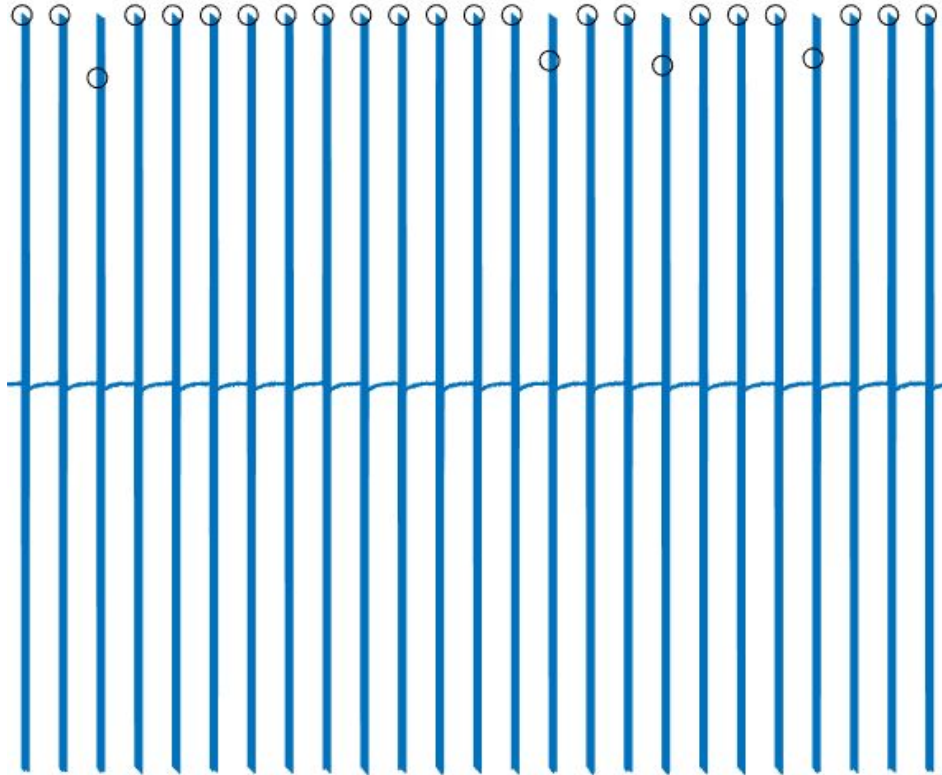
# Appendix

# Stimulation Experiments

## 4th Order Butterworth Bandpass Filter 20 - 450 Hz



# Data Extraction for Analysis

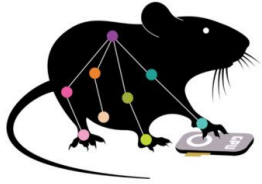


Stimulation Channel - depicts pulse delivery

For each stimulation experiment:

- Recorded several trains
- Amplitude thresholding to isolate start times of trains
- Slice data to create dataset for ML model.

## Tools We Used:



**DeepLabCut<sup>TM</sup>:**  
a software package for  
animal pose estimation



# Logistic Regression Analysis

- Performed simple logistic regression (using scikit-learn) on our collected data
- 50 data points collected in total, 80-20 split for training and testing
- **100% test accuracy suggests there is significant measurable difference between relaxing and resisting**
  - Binary classification task may just be insanely easy in this case
  - Small data set is our main source of skepticism for this result

```
==== RUNNING WITH RANDOM SEED 42 ====
Initializing Logistic Regression model. Attempting to train with data from: ./data2
Loading all dataset trials from ./data2...
Processing dir: class0, label: 0, expecting .mat var: 'trials'
  Loading from .mat file: RelaxStimTrials.mat
    Found 26 trials in RelaxStimTrials.mat using var 'trials'.
Processing dir: class1, label: 1, expecting .mat var: 'StimContract'
  Loading from .mat file: contractStim.mat
    Found 24 trials in contractStim.mat using var 'StimContract'.

Performing fixed split with 20 samples per class for training...
Split summary:
  Train set: 40 samples (20 class 0, 20 class 1)
  Test set: 10 samples (6 class 0, 4 class 1)
Logistic Regression model trained with 40 data points.
  Class 0: 20 samples
  Class 1: 20 samples
Test set has 10 data points.
  Class 0: 6 samples
  Class 1: 4 samples

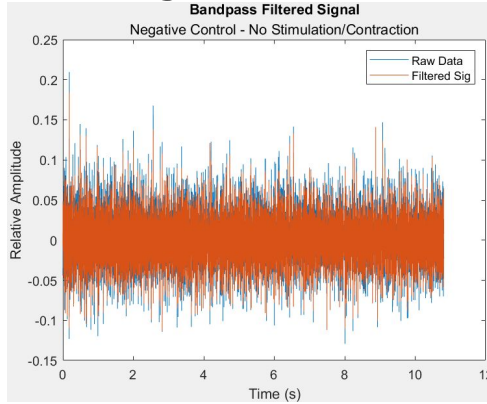
--- Evaluating Model on Held-Out Test Split ---
Test set size: 10 samples.
Class distribution in test set: Class 0: 6, Class 1: 4
Misclassification error: 0.0000
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1 Score: 1.0000
Confusion Matrix:
  TN: 6, FP: 0
  FN: 0, TP: 4
```

# Data Pre-Processing

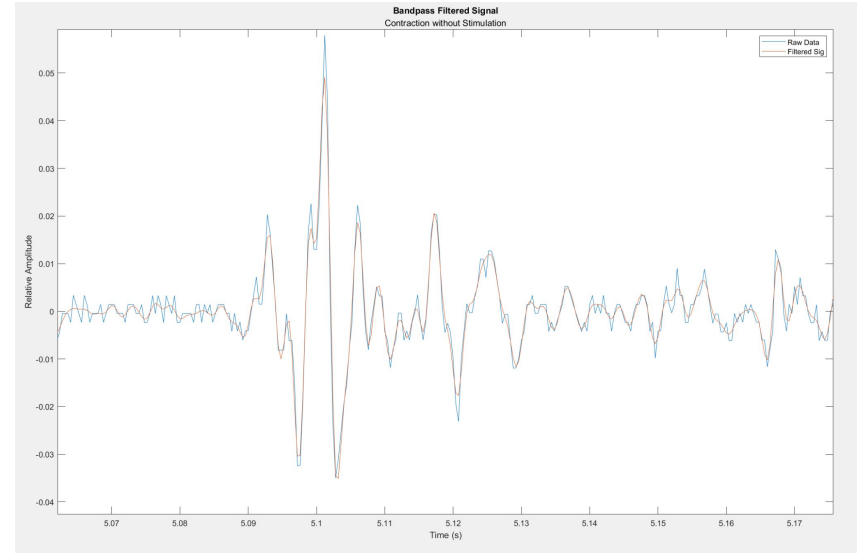
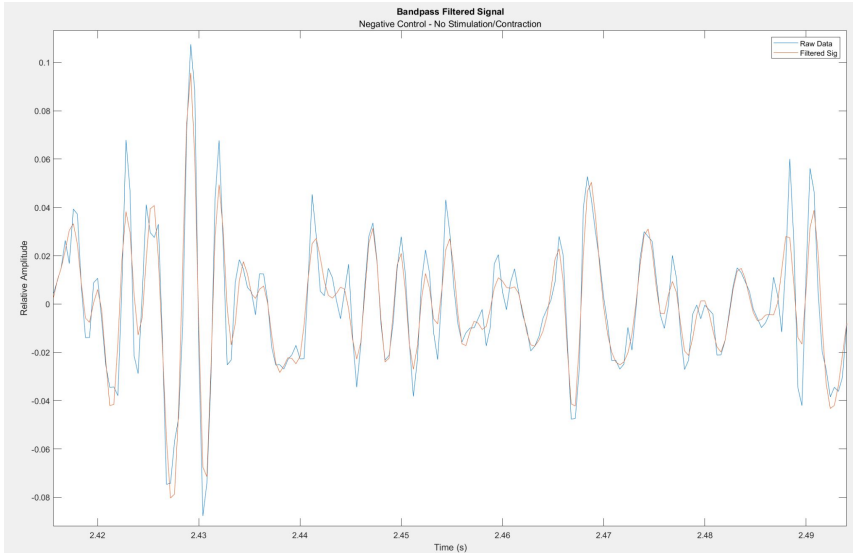
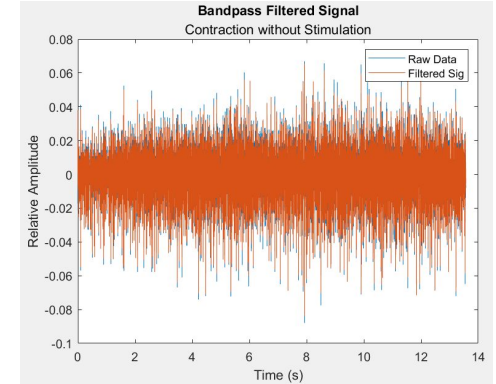
4th Order Butterworth Bandpass Filter 20 - 450 Hz

Negative Control:  
No Stimulation  
No Contraction

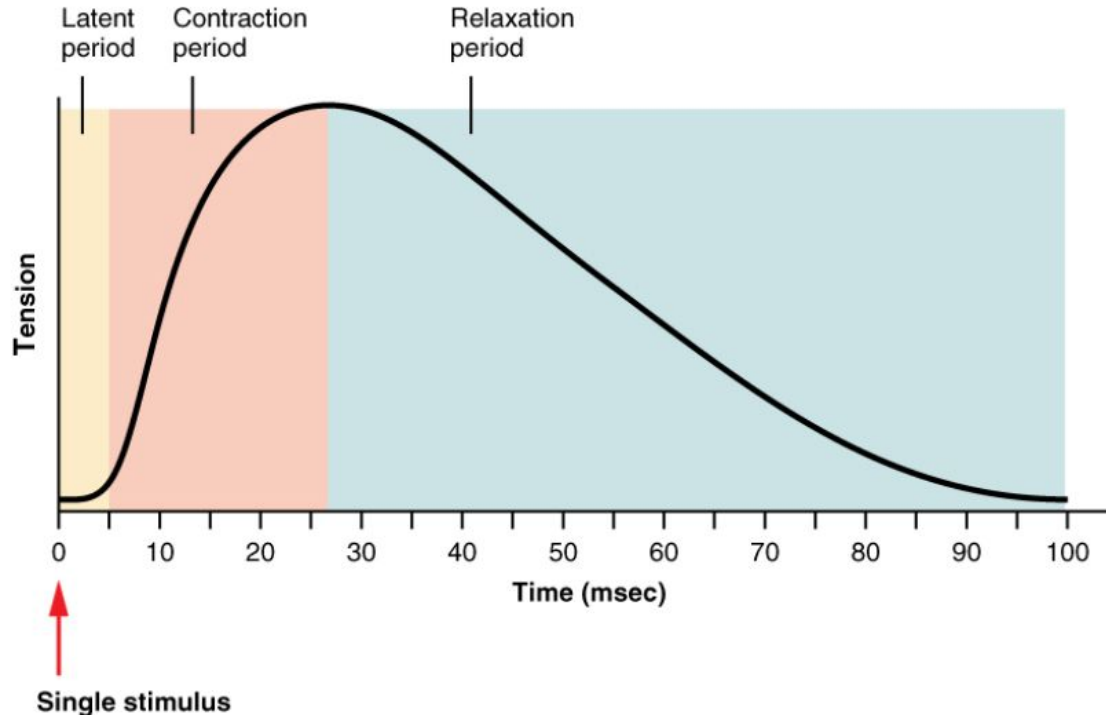
Zoomed-In View



Contraction  
without  
Stimulation



# Anatomy of muscle twitch

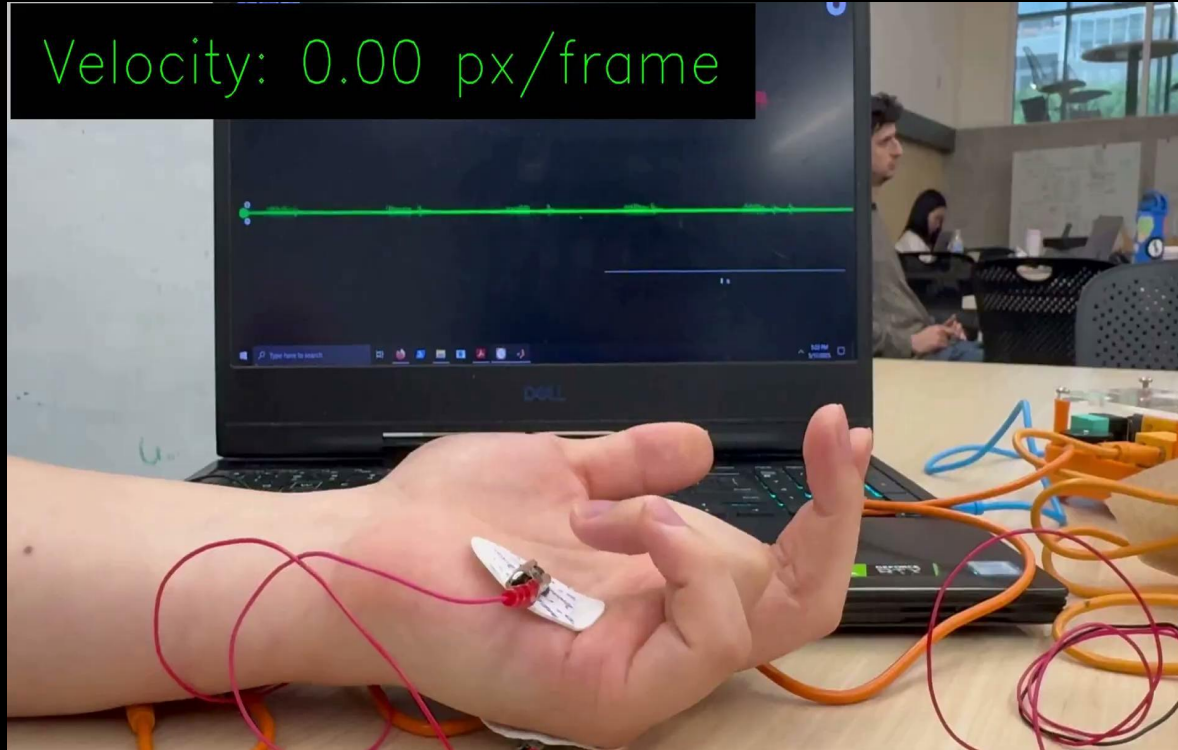


# Imagine...

**Frustration can arise when using technology that requires sensory input, especially when it comes to movement that cannot be controlled (e.g., a person with involuntary twitches operating a mouse)**

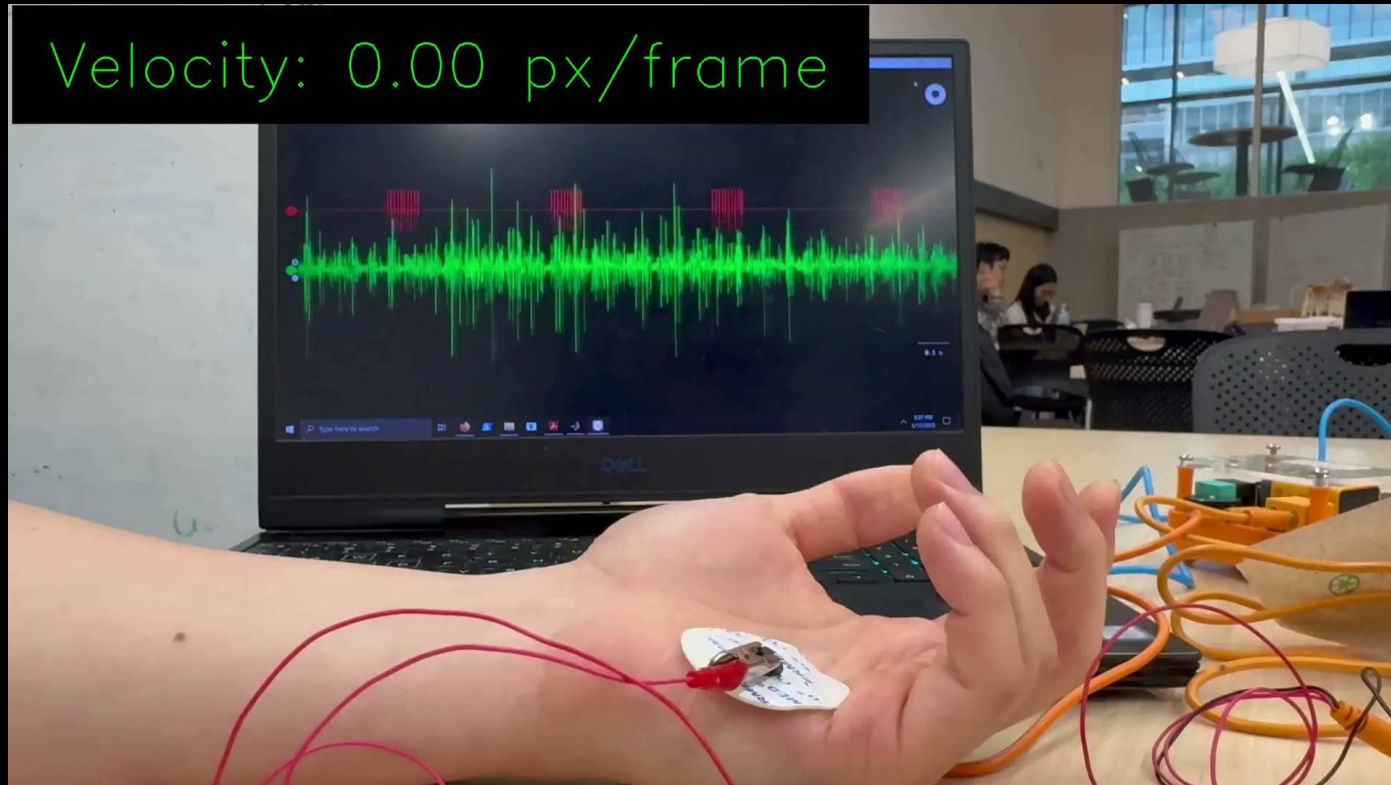
- **Main research q:** Can we distinguish intentional movement from involuntary movement, even under similar motor output?
- This can apply to accessibility technology that allows individuals to communicate intent, interact with digital environments, or control assistive devices using subtle, low-fatigue movements...even when traditional input methods like touch or speech aren't viable

# Stimulation | No Resistance

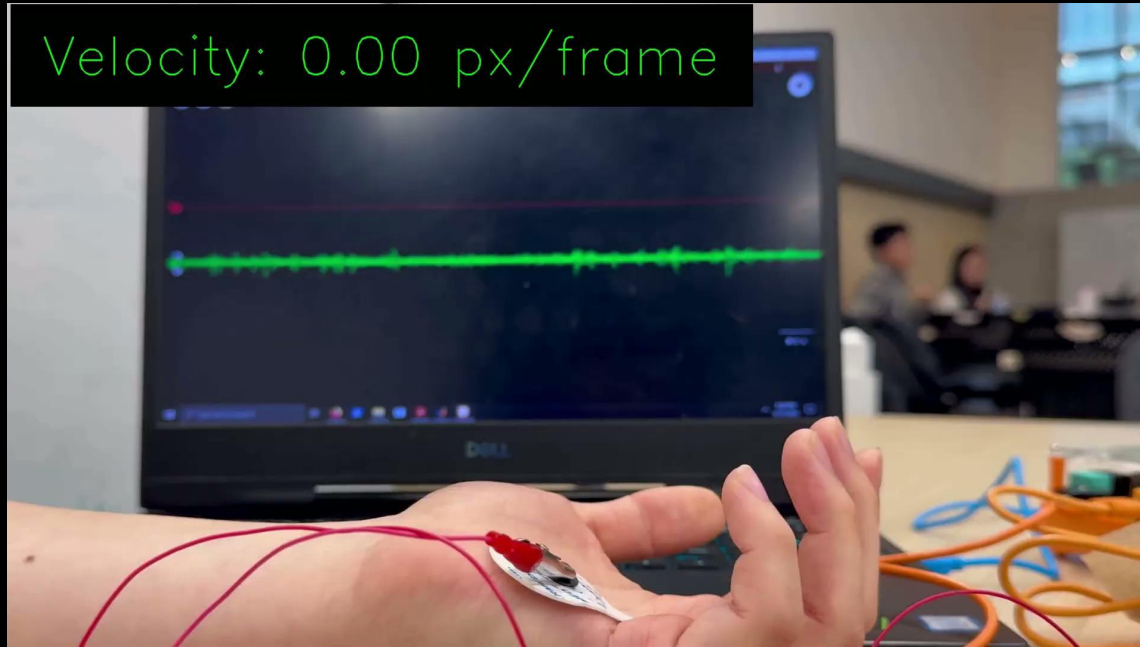


- Using velocity as a quantitative proxy to confirm whether or not the movement occurred

# Stimulation | Isometric Resistance



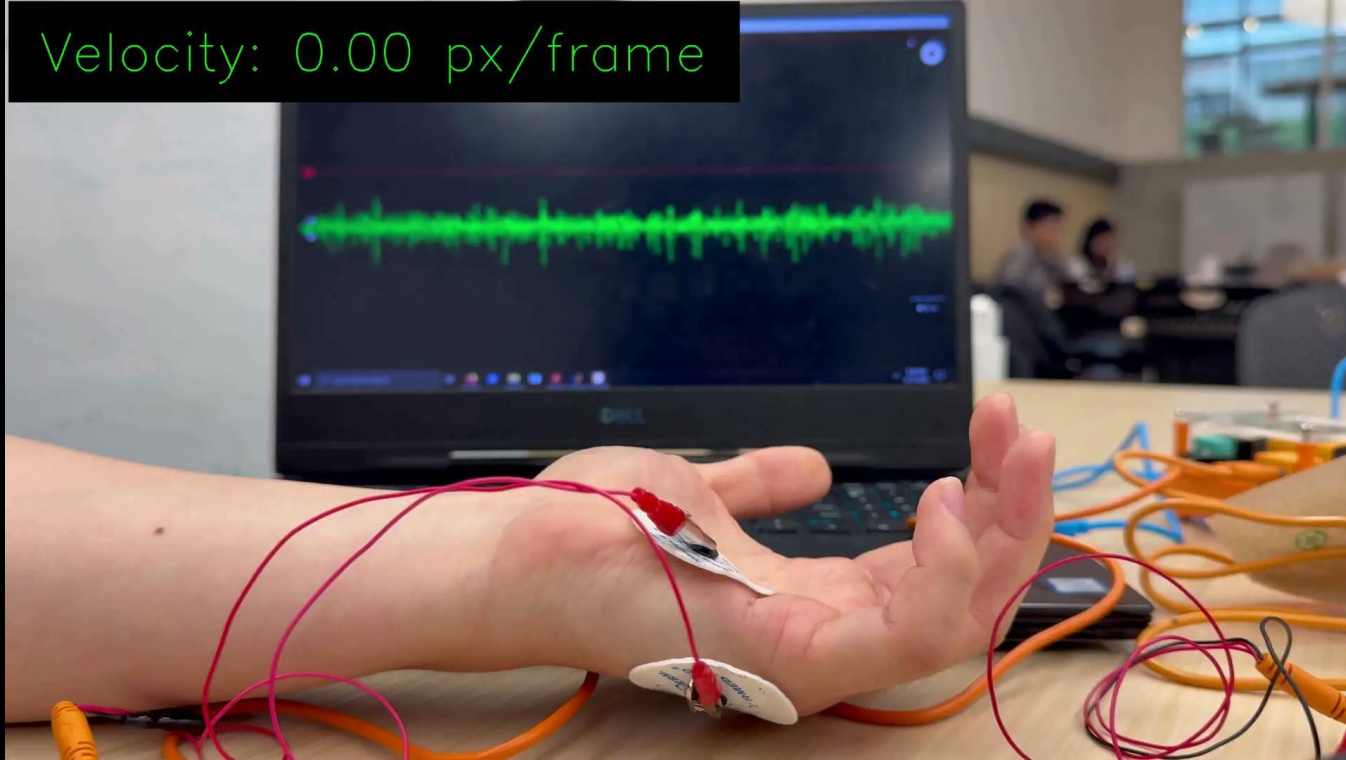
No Stimulation | No Resistance



No Stimulation | No Resistance

# No Stimulation | Resistance

Velocity: 0.00 px/frame



# Code (Matlab)

```

1 %% Attempt at Fourier Domain Filtering - too computationally expensive
2 downFactor = 4;
3 newdata = downsamples(data, downFactor);
4 freqDom = abs(fft(newdata));
5
6 Fs = 10000 / downFactor;
7 N = length(newdata);
8 timeAxis = (0:N-1) / Fs;
9 % f = (0:N-1)*(Fs/N);
10 f = linspace(0, Fs/2, N/2);
11 lower = 20;
12 upper = 450;
13 filterMask = (f >= lower) & (f <= upper);
14
15 filteredFreqDom = freqDom .* filterMask;
16 filteredSignal = ifft(filteredFreqDom, 'symmetric');
17
18 figure();
19 plot(timeAxis, newdata)
20 hold on
21 plot(timeAxis, real(filteredSignal))
22 hold off
23 legend(["Raw Data" "Filtered Signal"])
24 title('Fourier Transformed Filtering')
25 xlabel('Time (s)');
26 ylabel('Amplitude');

```

```

29 %% Butterworth filtfilt
30 % Parameters and downsampling
31 downFactor = 4;
32 newdata = downsamples(data, downFactor);
33 Fs = 10000 / downFactor;
34 N = length(newdata);
35 timeAxis = (0:N-1) / Fs;
36 lower = 20;
37 upper = 450;
38
39 [b, a] = butter(4, [lower, upper] / (Fs/2), 'bandpass');
40 newFilteredSignal = filtfilt(b, a, newdata);
41 standardizedSignal = (newFilteredSignal - mean(newFilteredSignal)) / std(newFilteredSignal);
42
43 figure();
44 plot(timeAxis, newdata)
45 hold on
46 plot(timeAxis, StandardizedSignal)
47 hold off
48 legend(["Raw Data" "Filtered Sig" "stan signal"])
49 title("Bandpass Filtered Signal - Negative Control")
50

```

```

51 %%
52 % EMGSignalFilter(relaxedNoStim, "Negative Control - No Stimulation/Contraction")
53
54 EMGSignalFilter(contractNoStim, "Contraction without Stimulation")
55
56 % EMGSignalFilter(stimNoContract, "No Contraction w/Stimulation")
57

```

```

58 %% Stimulation Data Processing
59 stimulationChannel = stimContract(:,2);
60 stimulationRecording = stimContract(:,1);
61
62 % Finds the indices where the amplitude is greater than average (when
63 % pulses were delivered). Extracts the times between extracted pulses.
64 stimThreshold = 0.7;
65 [~, locs] = findpeaks(stimulationChannel, 'MinPeakHeight', stimThreshold);
66 pulseIntervals = diff(locs);

```

```

67
68
69 % Pulse groups have constant frequency -> processing steps to remove
70 % non-pulses with similar amplitudes based on distance between pulses.
71 stimIndices = [];
72 inBurst = false;
73
74 % Saves stimulation indices
75 for idx = 1:length(locs)
76     spikeID = locs(idx);
77     if idx > length(pulseIntervals)
78         return;
79     end
80     timing = pulseIntervals(idx);
81     if timing == 11 || timing == 10
82         stimIndices = [stimIndices, spikeID];
83         inBurst = true;
84     else
85         if inBurst
86             stimIndices = [stimIndices, spikeID];
87         end
88         inBurst = false;
89     end
90 end
91
92

```

```

93 %% Extracting Start of Each Train to separate into trials
94 trainIndices = [];
95 for stimIdx = 1:length(stimIndices)-1
96     if abs(stimIndices(stimIdx) - stimIndices(stimIdx + 1)) > 2000
97         trainIndices = [trainIndices, stimIndices(stimIdx+1)];
98     end
99 end
100
101 figure();
102 plot(stimulationChannel)
103 hold on
104 % plot(stimIndices, stimulationChannel(stimIndices), 'ro')
105 plot(trainIndices, stimulationChannel(trainIndices), 'ko')
106 hold off
107

```

```

108 %% Splitting trains into trials
109 trialLen = trainIndices(2) - trainIndices(1);
110 trials = cell(length(trainIndices)-1, 1);
111 for trial = 1:length(trainIndices)-1
112     start = trainIndices(trial);
113     term = trainIndices(trial + 1);
114     trialData = stimulationRecording(start:term);
115     trials{trial} = trialData;
116 end
117
118 figure;
119 hold on;
120 for i = 1:length(trials)
121     plot(trials{i});
122 end

```

```

1 function EMGSignalFilter(data, name)
2 data = data(:, 1);
3 downFactor = 4;
4 newdata = downsamples(data, downFactor);
5 Fs = 10000 / downFactor;
6 N = length(newdata);
7 timeAxis = (0:N-1) / Fs;
8 lower = 20;
9 upper = 450;
10
11 [b, a] = butter(4, [lower, upper] / (Fs/2), 'bandpass');
12 newFilteredSignal = filtfilt(b, a, newdata);
13 standardizedSignal = (newFilteredSignal - mean(newFilteredSignal)) / std(newFilteredSignal);
14
15 save('C:\Users\edric\OneDrive\Junior Year\standardizedSignal.mat', 'standardizedSignal')
16 figure();
17 plot(timeAxis, newdata)
18 hold on
19 plot(timeAxis, standardizedSignal)
20 hold off
21 legend(["Raw Data" "Filtered Sig"])
22 title("Bandpass Filtered Signal", name)
23 xlabel("Time (s)")
24 ylabel("Relative Amplitude")

```

# DeeplabCut Code

```
import cv2
import numpy as np
import pandas as pd

video_path = r'C:\Users\hryu673\Downloads\NEUROHACKATHON\Video4.mp4'
output_path = r'C:\Users\hryu673\Downloads\NEUROHACKATHON\Video4_speed_fixedrate.mp4'
csv_path = r'C:\Users\hryu673\Downloads\NEUROHACKATHON\Video4-Hannah-2025-05-18\labeled-data\Video4\CollectedData_Hannah.csv'

#Loading position data
df = pd.read_csv(csv_path, header=[0, 1, 2])
x = df[['Hannah', 'bodypart1', 'x']].dropna()
y = df[['Hannah', 'bodypart1', 'y']].loc[x.index]
velocity = np.sqrt(x.diff().fillna(0)**2 + y.diff().fillna(0)**2).values

cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
frame_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

#Frames per velocity entry
frames_per_velocity = total_frames // len(velocity)

#Writing vid
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # MP4 codec
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_w, frame_h))
frame_idx = 0
velocity_idx = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break
```

```
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Display same velocity value over fixed frame chunk
    if velocity_idx < len(velocity):
        v = velocity[velocity_idx]
        if frame_idx % frames_per_velocity == 0 and frame_idx != 0:
            velocity_idx += 1
            if velocity_idx >= len(velocity):
                v = velocity[-1]
        else:
            v = velocity[-1]

    #Overlay
    velocity_text = f"Velocity: {v:.2f} px/frame"

    font = cv2.FONT_HERSHEY_SIMPLEX
    font_scale = 5.0
    thickness = 4
    padding = 90

    text_size, _ = cv2.getTextSize(velocity_text, font, font_scale, thickness)
    text_w, text_h = text_size
    text_x, text_y = 100, 200

    cv2.rectangle(
        frame,
        (text_x - padding, text_y - text_h - padding),
        (text_x + text_w + padding, text_y + padding),
        (0, 0, 0),
        thickness=-1
    )
```